Finding failures in supercomputer logs

LogBERT versus a rule-guided LLM, compared under one fair setup

Seven real lines from the BGL log. Orange: alerts.

```
-        instruction cache parity error corrected
-        generating core.304
-        1347195 double-hummer alignment exceptions
KERNDTLB data TLB error interrupt
-        ciod: LOGIN chdir(…) failed: Permission denied
APPOUT   ciod: LOGIN chdir(…) failed: Input/output error
-        10732 total interrupts. 0 critical input …
```

CSCI 390 · Natural Language Processing and Applications · Dr. Qi Li

github.com/Amshu-0/claude-nlp

What a log line contains

KERNDTLB	1118536327	R30-M0-N9-C:J16-U01	RAS	KERNEL	FATAL	data TLB error interrupt
alert tag	time	node	type	component	level	content

The alert tag is the label

Administrators marked every alert with one of 41 categories; '-' means normal. The pipeline cuts the tag off before any model reads the line.

The content is what the models read

LogBERT sees it as a template ID; the language model reads its words.

4,747,963

log lines, June 2005 to January 2006

348,460

alerts: 7.3% of all lines

BlueGene/L

supercomputer, Lawrence Livermore National Laboratory

Why this is hard

7.3%

Alerts are rare

Always answering 'normal' is 92.7% accurate and useless. So we measure precision, recall and F1.

79.6%

The log changes

of the test messages use a template never seen in training.

error ≠ alert

Words mislead

'instruction cache parity error corrected' is normal. 'LOGIN … failed: Input/output error' was an alert.

Research question

How does a BERT-style model that learns sequences of log-template IDs compare with a rule-guided large language model that reads the log text itself, in identifying anomalous system events?

Two approaches, one fair setup

A LogBERT

Guo et al., 2021 · reproduced

Reads	sequences of template IDs in a 5-minute window; never the words
Learns from	normal windows only; it never sees an alert
Model	a small Transformer, 1.67 M parameters, trained from scratch
Flags	a window whose hidden keys it cannot predict

B Rule-guided LLM

LogRules, Huang et al., 2025 · adapted

Reads	the words of each template; never the sequence
Learns from	labelled training templates, turned into verified rules
Model	Qwen2.5-7B-Instruct, 7.6 B parameters, used as it is
Flags	templates the rules (or the LLM applying them) call abnormal

Shared setup

same BGL data · chronological 60 / 20 / 20 split · the same 5,742 test windows · one scoring script · label removed first

From log lines to templates and windows

Drain groups messages into templates

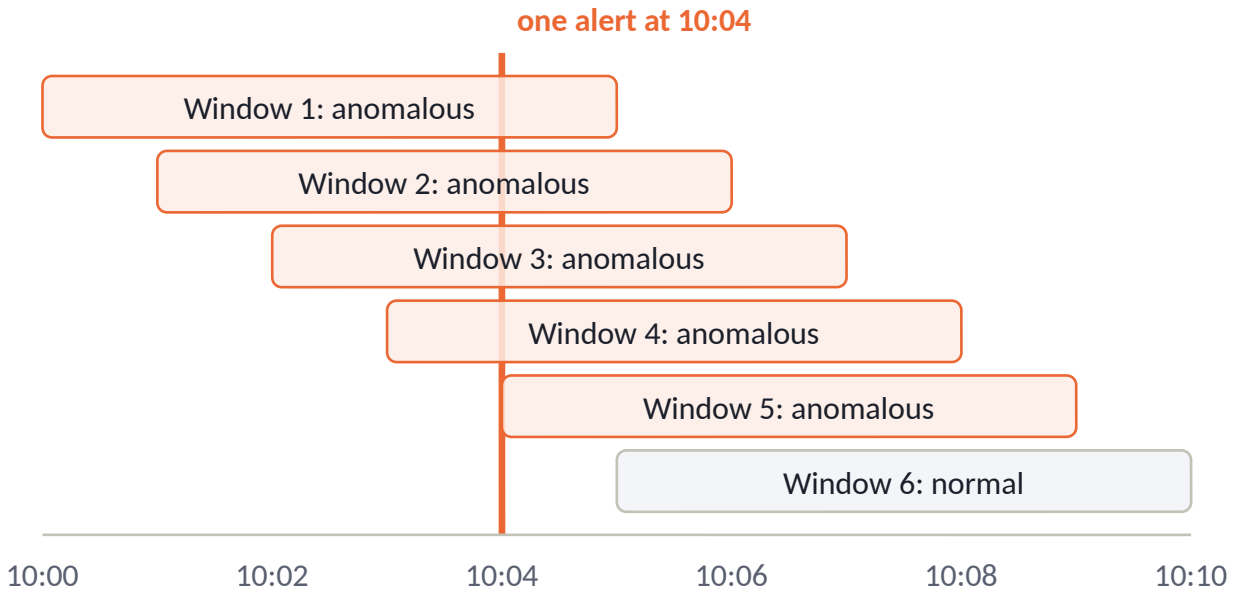
```
generating core.304
generating core.2276
generating core.5089
→ generating core.<*>

ciod: LOGIN chdir(/p/gb1/…/2183)
      failed: Input/output error
→ ciod: LOGIN <*> failed: <*> <*>
```

4.7 M → 923

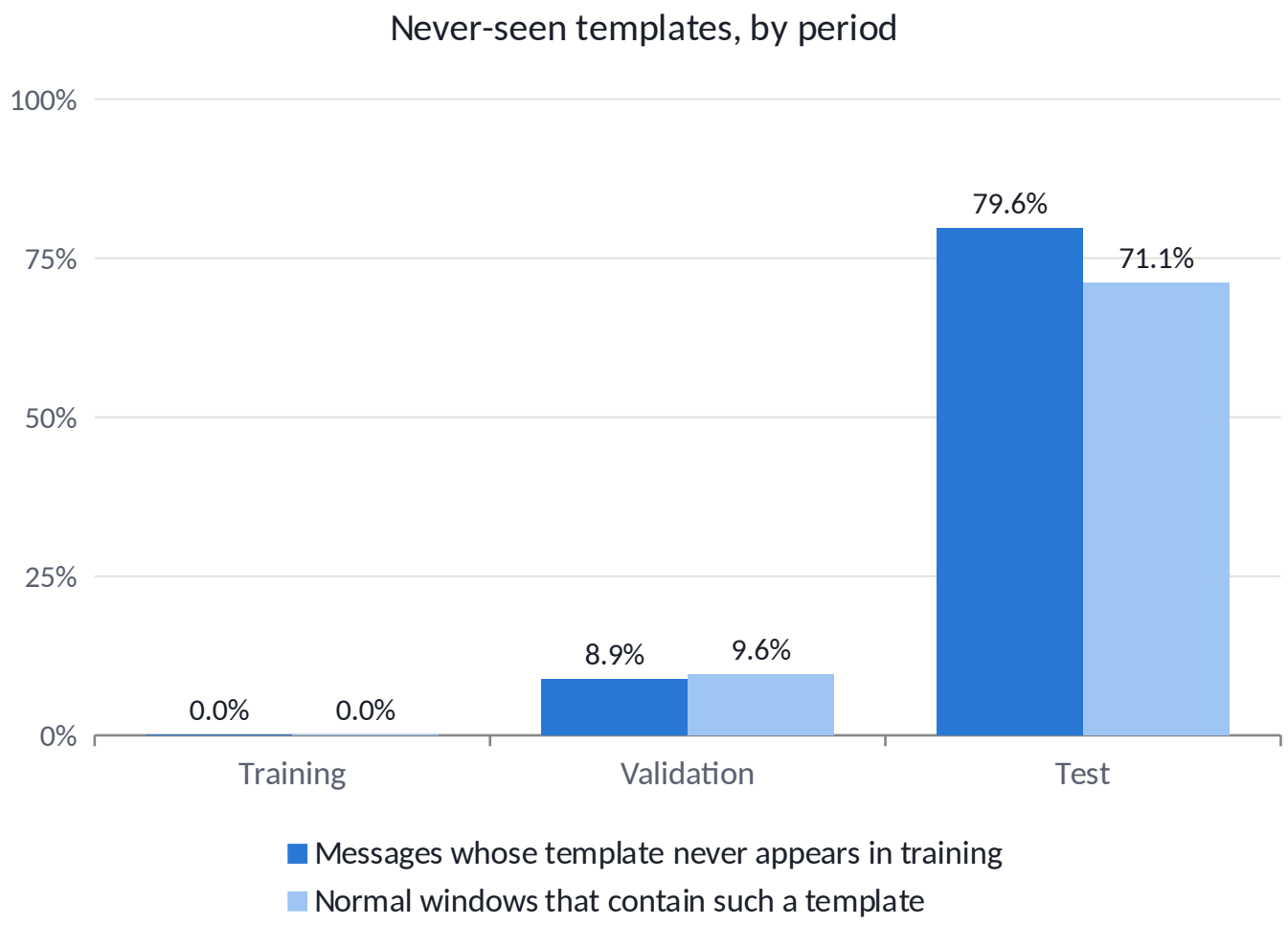
messages become 923 templates; each template ID is a 'log key', like a word in a vocabulary

Windows: 5 minutes, sliding by 1 minute



- A window is anomalous if it contains at least one alert.
- One alert makes up to five overlapping windows anomalous.
- Test period: 5,742 windows, 495 of them anomalous (8.6%).

The log changes over time



Split by time, never shuffled

Train on the first 60% of the timeline (June to October 2005), tune on the next 20%, test on the last 20% (November 2005 to January 2006). A random split would let a model learn from the future.

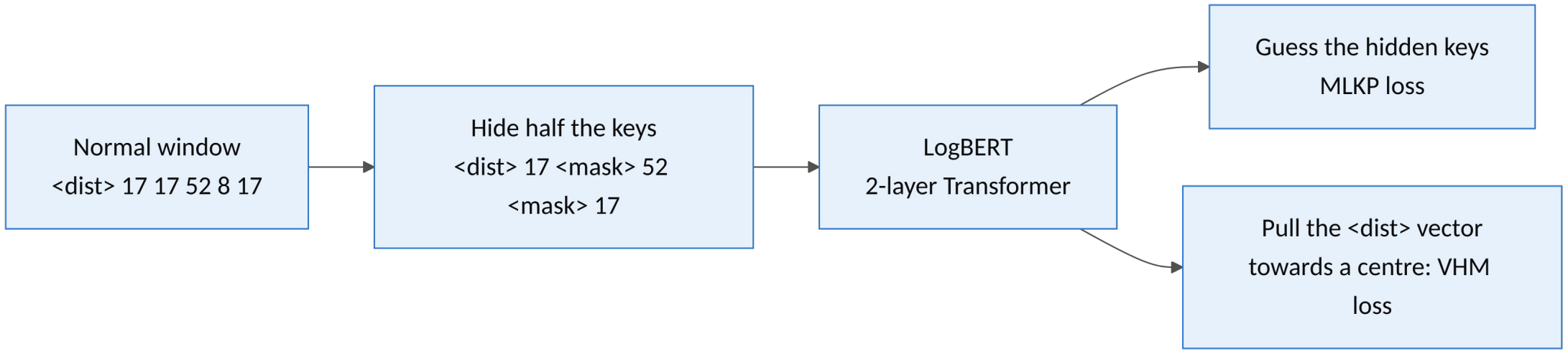
Period	Messages	Windows
Training	3,569,472	21,112
Validation	696,160	10,478
Test	482,331	5,742

New in December

Two periodic diagnostics appear: 135,092 test messages each, never seen before October 2005.

LogBERT learns “normal” by filling in blanks

A



Normal windows only

19,029 normal training windows, 11,615 distinct sequences. LogBERT never sees an alert.

Two losses

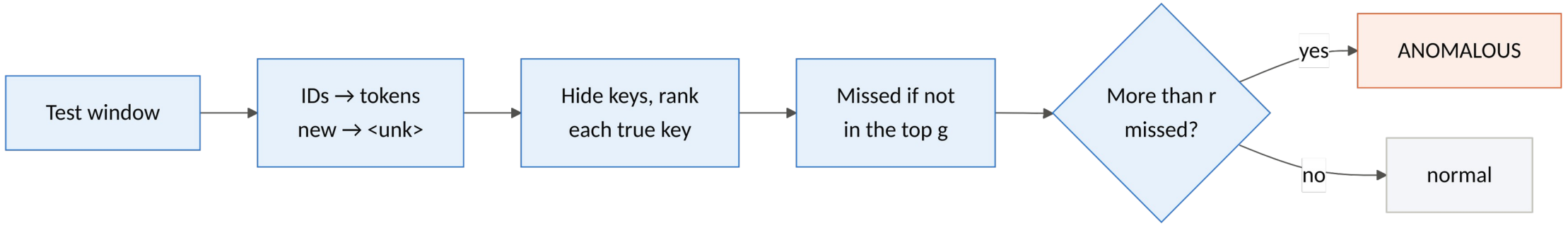
Predict the hidden keys (MLKP) and pull each sequence's summary vector towards one centre (VHM): loss = MLKP + 0.1 × VHM.

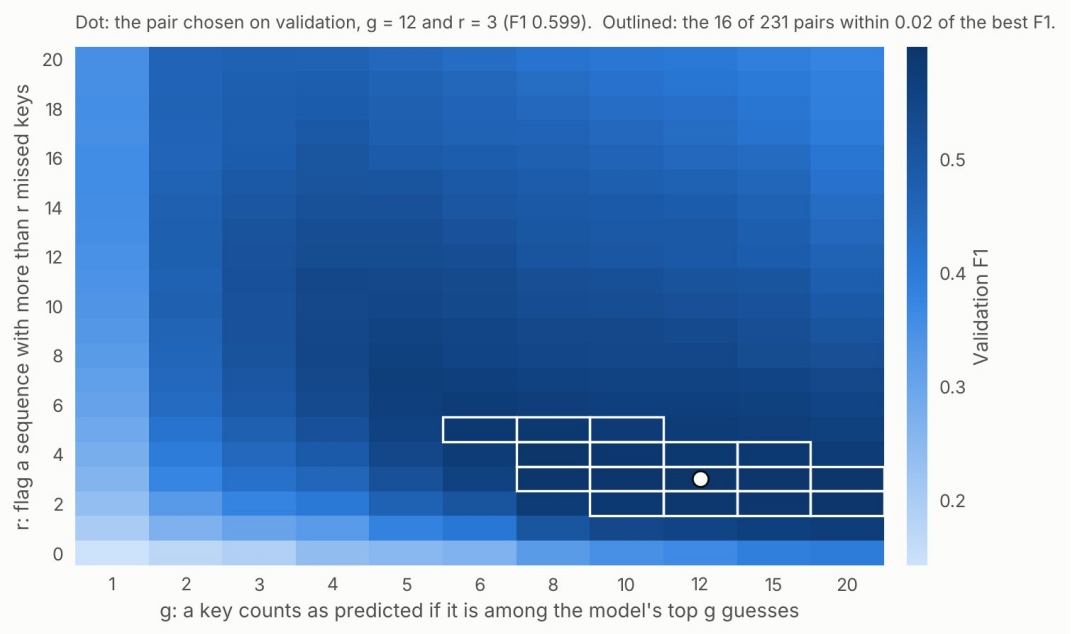
A small Transformer

256-number embeddings, 2 self-attention layers with 4 heads: 1.67 M parameters. 60 minutes on 4 CPU cores.

LogBERT flags windows it cannot predict

A





- g and r are chosen on the validation period: g = 12, r = 3 (validation F1 0.599).
- The choice is fragile: only 16 of 231 pairs come within 0.02 of the best.
- The original code tunes on the test set. Here that would report F1 0.475 instead of 0.293.
- Our extension A+ checks every key once instead of a random half: F1 0.305.

One LLM, four kinds of guidance

B

B1 Zero-shot

The template and a question: abnormal or normal?

B2 Few-shot

Plus 8 labelled training templates as examples.

B3 Rule-based

Plus the 30 verified rules. The model must cite the rule it applied.

B4 Keyword rules

The same 30 rules as plain string matching. No LLM: the control experiment.

The model

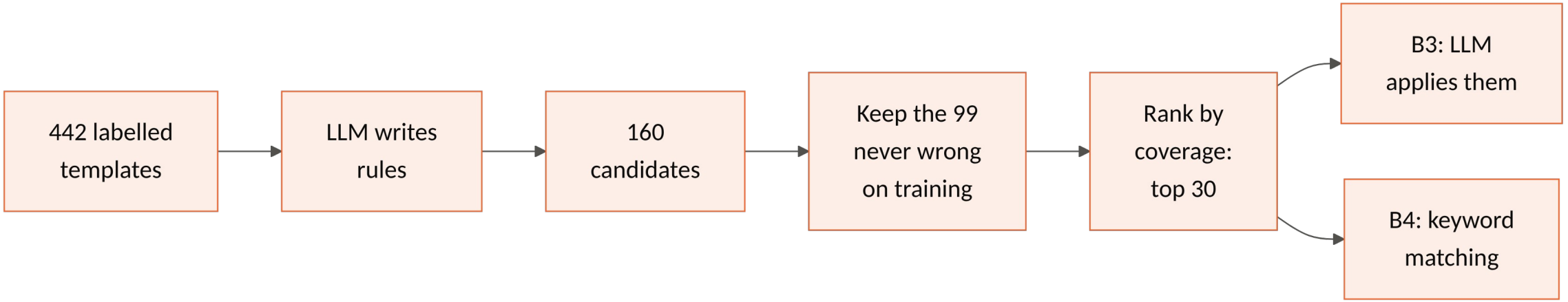
- Qwen2.5-7B-Instruct, 7.6 B parameters, 4-bit (4.7 GB)
- llama.cpp on 4 CPU cores, temperature 0: reproducible
- 6 to 7 seconds per template; every answer cached
- Each of the 472 test templates classified once; the label is copied to all its messages

A real B3 answer

```
Log: <*> TLB error interrupt
{"label": "abnormal", "rule": 24,
 "reason": "TLB error interrupt is
            typically abnormal"}
```

The LLM writes rules; the data checks them

B



A verified rule (rank 4 of 99)

“Messages mentioning 'FAILED', 'terminated for reason', 'kernel panic', and 'machine check interrupt' are indicative of system failures.”

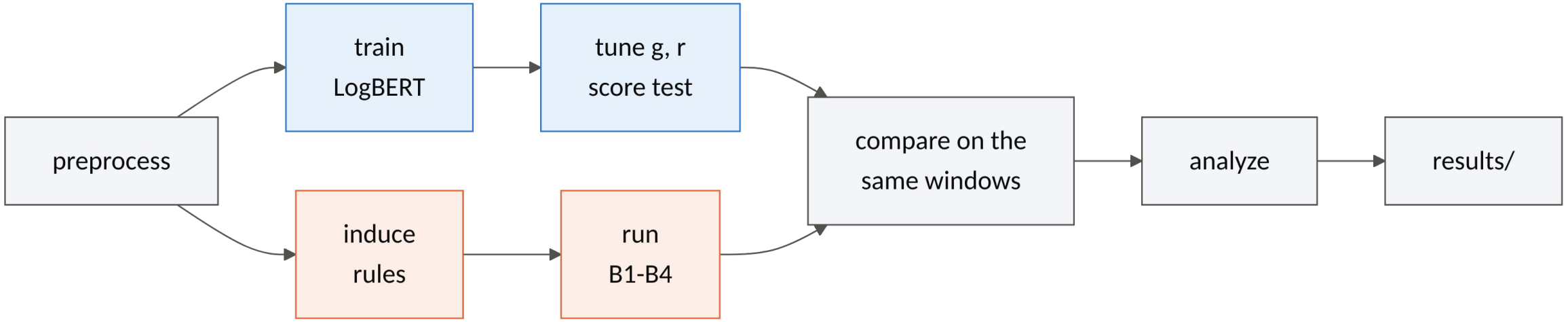
Keywords: lustre mount failed · rts: kernel terminated for reason · kernel panic · machine check interrupt

Correct on 9 of 9 matching training templates.

Weak spots, visible already

- Vague sentences: “Indicates a system error or failure.”
- Near-duplicates: rules 8, 9 and 10 all key on ‘terminated’.
- Odd keywords that happen to pass: ‘pro’, ‘<*>:<*>eeeeeee’.
- Verification checks the keywords, not the sentence the model reads.

One pipeline, tested end to end



28

Python modules in one package, logad: data, logbert, llm, rules, evaluation

1

YAML configuration for the whole experiment; any value can be overridden

148

automated tests, offline, about 10 s; CI on every push

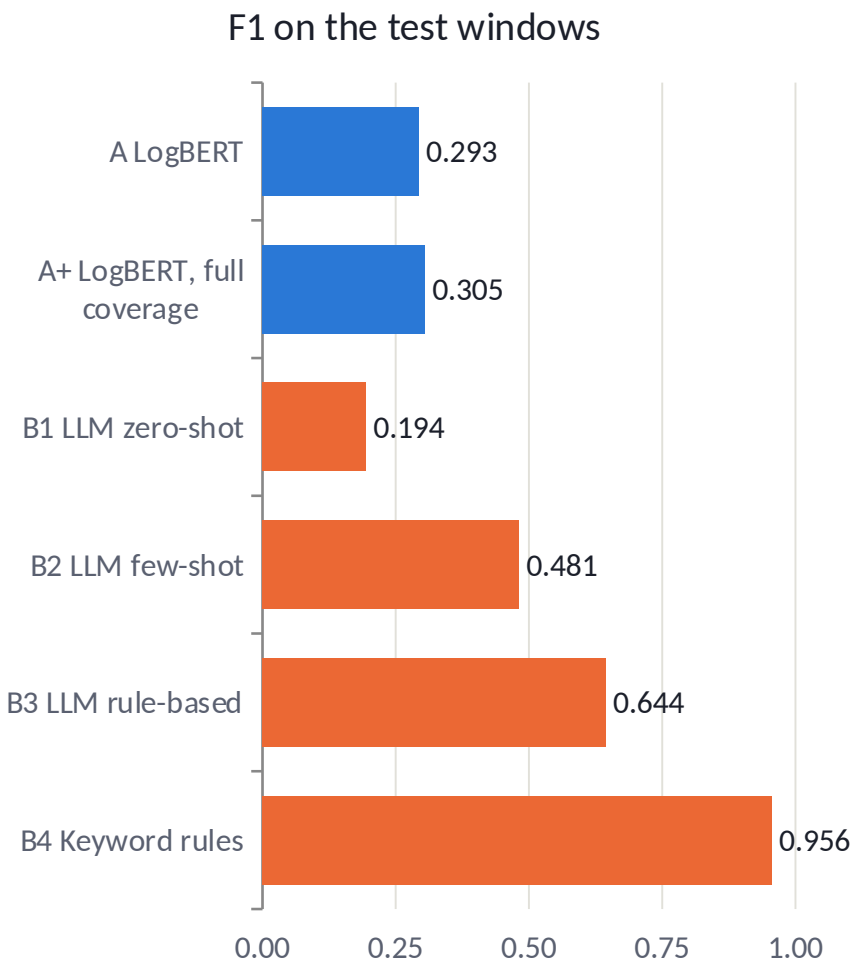
1,458

LLM calls, all cached and committed: Approach B replays without a model

Results: the rules win, without the LLM

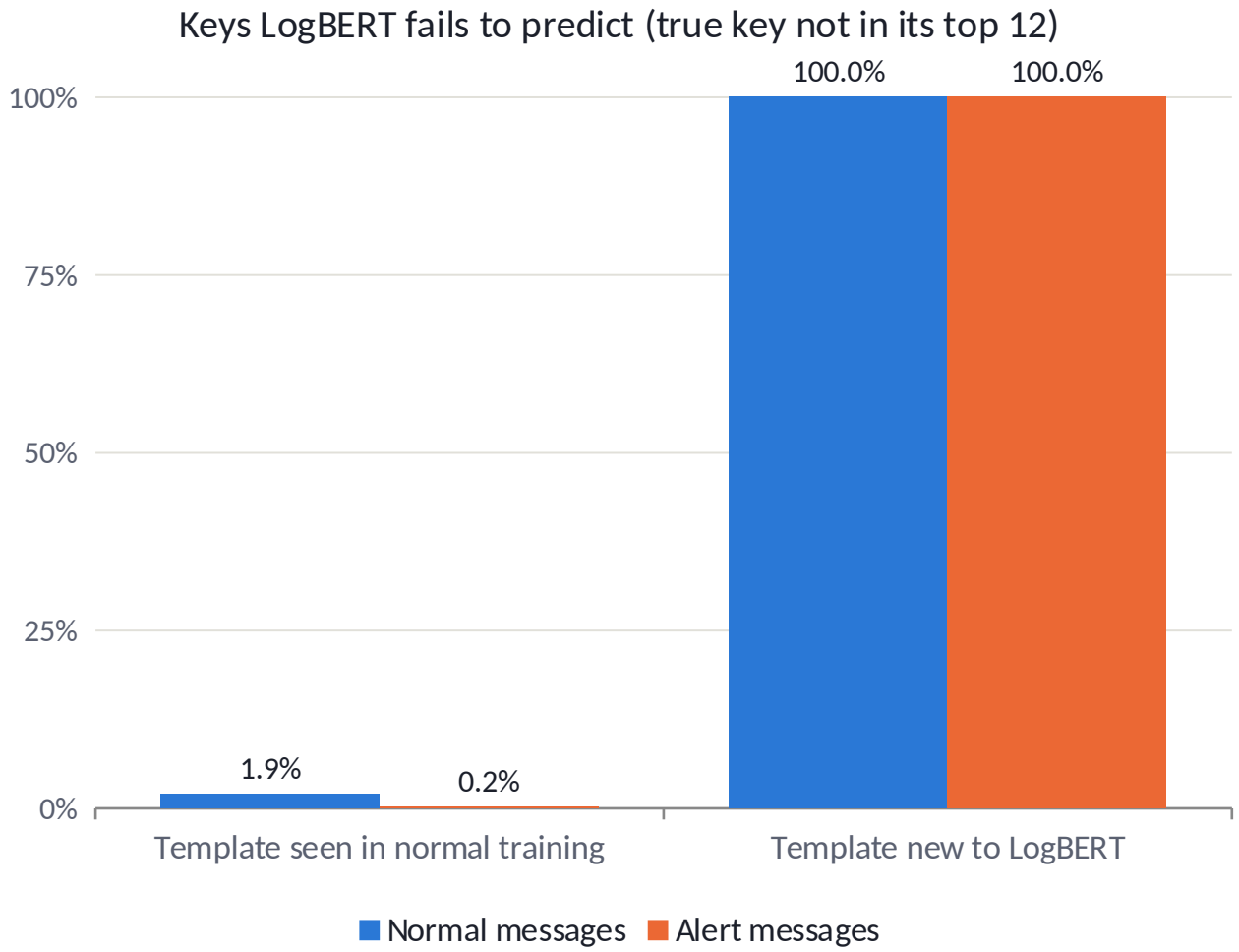
Method	Precision	Recall	F1	False alarms	Time
A LogBERT	0.174	0.921	0.293	2,159	78 s
A+ LogBERT, full coverage	0.184	0.897	0.305	1,975	156 s
B1 LLM zero-shot	0.108	1.000	0.194	4,103	47 min
B2 LLM few-shot	0.323	0.941	0.481	976	53 min
B3 LLM rule-based	0.489	0.943	0.644	488	57 min
B4 Keyword rules, no LLM	**1.000**	**0.915**	**0.956**	**0**	**8 ms**

Test period: 5,742 windows, 495 anomalous (8.6%). Time: a verdict for every test window on 4 CPU cores. The best any template-level method could reach: F1 0.996. In the chart, blue is Approach A and orange is Approach B.



LogBERT flags whatever is new

A



97%

of the alert keys come from templates LogBERT never saw: it detects newness, not failure

57.7% vs 0.3%

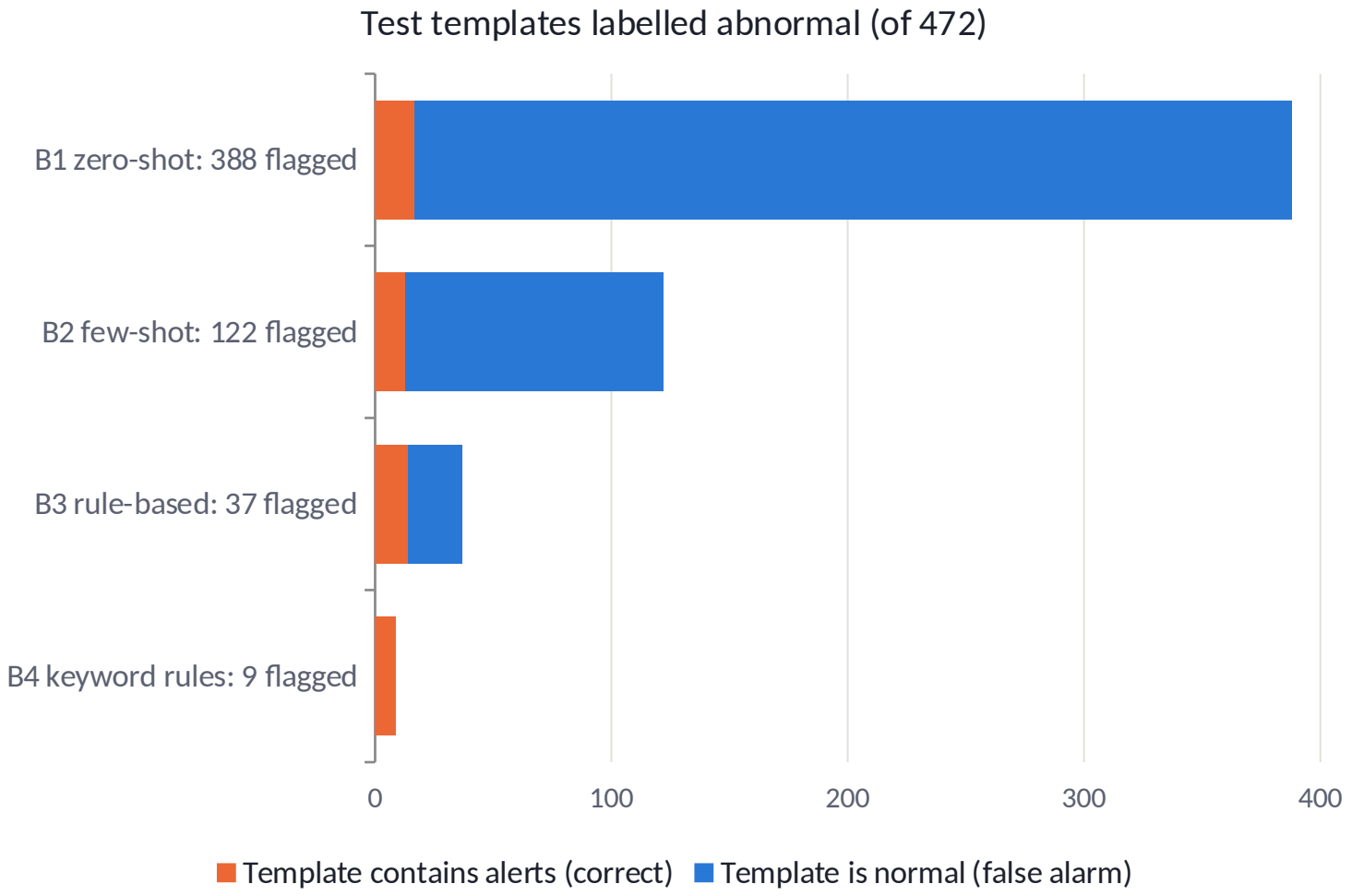
of normal test windows flagged, with and without a never-seen template

99.8%

of its 2,159 false alarms contain a never-seen template

Guidance narrows the LLM

B



20

of the 472 test templates contain alerts

- Zero-shot, the LLM calls anything error-like abnormal: 388 templates.
- Examples cut this to 122, rules to 37: guidance helps, as LogRules found.
- The keyword rules flag 9 templates, and all 9 contain alerts.

The LLM follows rule wording, not keywords

B

20 of 420

rule citations where the cited rule's keyword actually occurs in the log

Log (normal: 27,510 messages)

```
ciod: Received signal <*>, code=<*>,
      errno=<*>, address=<*>
```

The LLM (B3) answers

```
{"label": "abnormal", "rule": 8,
 "reason": "contains 'terminated'"}
```

Rule 8 keywords: terminated · panic! - stopping execution · exited abnormally. None occurs in the log.

+5 alert templates

that no keyword matches, caught by reading the sentences: 14 more windows

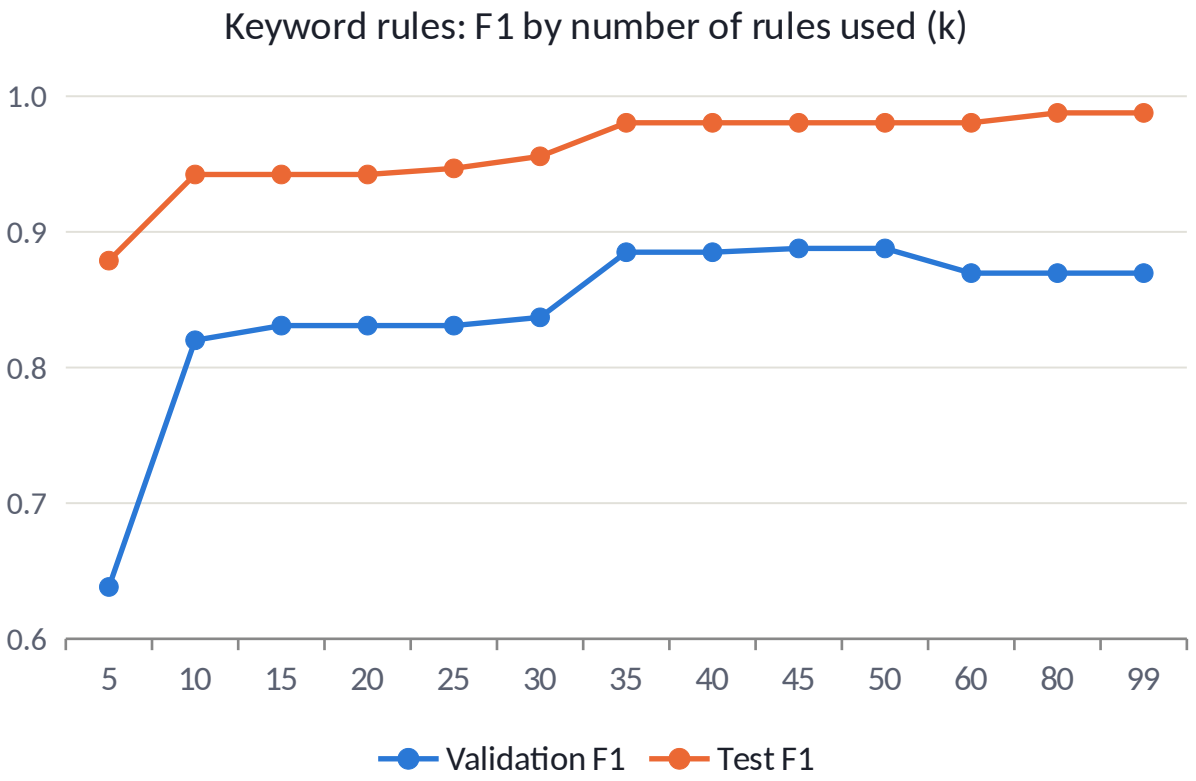
+23 false alarms

templates covering 57,309 normal messages; 12 of them cite the broad rule 4

0 false alarms

for the same rules as plain keyword matching (B4)

Two traps: rule ranking and label drift



k = 30, fixed in the proposal: test F1 0.956. k = 45, best on validation: 0.980. Near-duplicates fill the top 30, and the 'socket closed' rule sits at rank 32.

Label drift: one template, two labels

Training: normal, 472 messages

```
ciod: LOGIN chdir(…) failed:
      Permission denied
```

Test: alert, 816 messages

```
ciod: LOGIN chdir(…) failed:
      Input/output error
```

Drain: ciod: LOGIN <*> failed: <*> <*>

The words that decide the label were masked, so every method that learned from training calls it normal.

What went wrong, and how we fixed it

Reference code tunes on test

We tune g and r on validation. Tuning on test would report F1 0.475 instead of 0.293.

Shortcuts in the original pipeline

It shuffles windows, skips short test windows and truncates long ones. We split by time, score every window and chunk long ones.

No GPU, no API key

Qwen in 4-bit through llama.cpp on 4 CPU cores. Ending every prompt with the log line made calls 20% faster.

The machine restarted mid-run

Every LLM answer is cached, so the rerun reused 689 stored answers and computed only the rest.

Dataset host unreachable

We verified a mirror: checksum, 4,747,963 lines, 348,460 alerts, all 2,000 sample lines present.

Small samples mislead

On the 2,000-line sample LogBERT flags almost everything. Only the full log gives meaningful numbers.

Lessons and next steps

Lessons

1. The evaluation setup can change the answer more than the model.
2. Check what a model relies on: LogBERT turned out to detect newness.
3. Check that an LLM does what it says it does: 20 of 420 citations held.
4. Always run the simple baseline: here it won.

Next steps

- Select rules for the new templates they cover, not raw usage.
- Check rule sentences too, or show the LLM only the keywords.
- Train LogBERT longer on a GPU, and try other Loghub datasets (Thunderbird, Spirit).
- Combine A and B3: requiring both raises B3's F1 from 0.644 to 0.776.

Live demo

```
python app/demo.py --config configs/bgl_cpu.yaml --share
```

1. **`KERNDTLB 1118536327 2005.06.11 R30-M0-N9-C:J16-U01 … FATAL data TLB error interrupt`**
   The tag is removed first. Keyword rule 24 and the LLM flag it; LogBERT misses the key, but needs more than 3 misses.

2. **`instruction cache parity error corrected`**
   Says 'error' but is normal: the rules leave it alone. Zero-shot, the LLM called it abnormal.

3. **`ciod: LOGIN chdir(/p/gb1/stella/RAPTOR/2183) failed: Input/output error`**
   An alert hidden by label drift: it maps to a template that was normal in training.

4. **`Tab 2: insert  rts panic! - stopping execution  into a normal window`**
   Move it around and watch LogBERT's missed-key count and the 'without your line' comparison.

Conclusion

0.956

F1 of the verified rules applied as keywords, against 0.293 for LogBERT, under one fair, chronological setup.

79.6%

of test messages are new. LogBERT, reading only IDs, flags newness; reading the words tells a harmless new message from an alert.

20 of 420

The verified rules do the work: the LLM applies them loosely, and keyword matching is better and 400,000× faster.

Limits: one dataset, one chronological split, one run per method on a CPU budget, and one 4-bit 7B model that both wrote and applied the rules. A stronger rule generator, verified rule sentences, or a dataset whose failures are less tied to particular words could change the balance.

Thank you. Questions?

Code, results and a step-by-step guide to every part of the project

github.com/Amshu-0/claude-nlp
docs/guide.md · docs/report.md · results/

In one sentence

Reading the words beats reading only the IDs when the log changes, and verified keyword rules beat the LLM that wrote them.